Program No: 1	**Date:** 10-07-2026

Program Title: Demonstration of Storage Classes in C

Description:
Demonstrates automatic (local), external (global), static, and register storage classes in C.
Author: Abhishek Vincent | **Roll No:** 03 | **Date:** 10-07-2026

Source Code:

```
#include <stdio.h>;

extern int global_variable;
int global_variable = 10;

void show_local_variable()
{
    auto int local_variable = 5;
    printf("Local variable: %d\n", local_variable);
}

void show_global_variable()
{
    printf("Global variable accessed through extern declaration: %d\n", global_variable);
}

void show_static_variable()
{
    static int static_variable = 1;
    printf("Static variable: %d\n", static_variable);
    static_variable++;
}

void show_register_variable()
{
    register int register_variable;
    for (register_variable = 1; register_variable &lt;= 3; register_variable++)
    {
        printf("Register variable: %d\n", register_variable);
    }
}
```

```
int main()
{
    printf("Storage class demonstration\n\n");
    show_local_variable();
    show_global_variable();

    printf("First call to the static variable function:\n");
    show_static_variable();
    printf("Second call to the static variable function:\n");
    show_static_variable();

    show_register_variable();
    return 0;
}
```

Output Screenshot / Text:

C/C++

```
Storage class demonstration

Local variable: 5
Global variable accessed through extern declaration: 10
First call to the static variable function:
Static variable: 1
Second call to the static variable function:
Static variable: 2
Register variable: 1
Register variable: 2
Register variable: 3
```

Program No: 2	**Date:** 14-07-2026

Program Title: Use a menu-driven program to insert, search, delete and sort elements in an array using functions (use global variables)

Description:
Implements a menu-driven program to perform insertion, searching, deletion, and sorting on an array using global variables and functions.
Author: Abhishek Vincent | **Roll No:** 03 | **Date:** 14-07-2026

Source Code:

```
#include <stdio.h>

#define MAX 100

int array[MAX];
int size = 0;

void display()
{
    int i;

    if (size == 0)
    {
        printf("Array is empty.\n");
        return;
    }

    printf("Array: ");
    for (i = 0; i < size; i++)
    {
        printf("%d ", array[i]);
    }
    printf("\n");
}

void insert_element()
{
    int value;
    int position;
    int i;

    if (size == MAX)
    {
        printf("Array is full.\n");
        return;
    }
```

```
    printf("Enter the value to insert: ");
    scanf("%d", &value);
    printf("Enter the position (1 to %d): ", size + 1);
    scanf("%d", &position);

    if (position < 1 || position > size + 1)
    {
        printf("Invalid position.\n");
        return;
    }

    for (i = size; i >= position; i--)
    {
        array[i] = array[i - 1];
    }

    array[position - 1] = value;
    size++;
    printf("Element inserted.\n");
}

void search_element()
{
    int value;
    int i;

    printf("Enter the value to search: ");
    scanf("%d", &value);

    for (i = 0; i < size; i++)
    {
        if (array[i] == value)
        {
            printf("Element found at position %d.\n", i + 1);
            return;
        }
    }

    printf("Element not found.\n");
}

void delete_element()
{
    int value;
    int position = -1;
    int i;

    if (size == 0)
    {
        printf("Array is empty.\n");
```

```
        return;
    }

    printf("Enter the value to delete: ");
    scanf("%d", &value);

    for (i = 0; i < size; i++)
    {
        if (array[i] == value)
        {
            position = i;
            break;
        }
    }

    if (position == -1)
    {
        printf("Element not found.\n");
        return;
    }

    for (i = position; i < size - 1; i++)
    {
        array[i] = array[i + 1];
    }

    size--;
    printf("First occurrence deleted.\n");
}

void sort_elements()
{
    int i;
    int j;
    int temporary;

    for (i = 0; i < size - 1; i++)
    {
        for (j = 0; j < size - i - 1; j++)
        {
            if (array[j] > array[j + 1])
            {
                temporary = array[j];
                array[j] = array[j + 1];
                array[j + 1] = temporary;
            }
        }
    }

    printf("Array sorted in ascending order.\n");
}
```

```
int main()
{
    int choice;

    do
    {
        printf("\n1. Insert\n");
        printf("2. Display\n");
        printf("3. Search\n");
        printf("4. Delete\n");
        printf("5. Sort\n");
        printf("6. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice)
        {
            case 1:
                insert_element();
                break;
            case 2:
                display();
                break;
            case 3:
                search_element();
                break;
            case 4:
                delete_element();
                break;
            case 5:
                sort_elements();
                break;
            case 6:
                printf("Program ended.\n");
                break;
            default:
                printf("Invalid choice.\n");
        }
    } while (choice != 6);

    return 0;
}
```

Output Screenshot / Text:

```
1. Insert
2. Display
3. Search
4. Delete
```

```
5. Sort
6. Exit
Enter your choice: 1
Enter the value to insert: 54
Enter the position (1 to 1): 1
Element inserted.

1. Insert
2. Display
3. Search
4. Delete
5. Sort
6. Exit
Enter your choice: 1
Enter the value to insert: 12
Enter the position (1 to 2): 2
Element inserted.

1. Insert
2. Display
3. Search
4. Delete
5. Sort
6. Exit
Enter your choice: 1
Enter the value to insert: 1
Enter the position (1 to 3): 3
Element inserted.

1. Insert
2. Display
3. Search
4. Delete
5. Sort
6. Exit
Enter your choice: 1
Enter the value to insert: 7
Enter the position (1 to 4): 4
Element inserted.

1. Insert
2. Display
3. Search
4. Delete
5. Sort
6. Exit
Enter your choice: 1
Enter the value to insert: 20
Enter the position (1 to 5): 5
Element inserted.

1. Insert
```

LEARN • SERVE • EXCEL
RAJAGIRI

```
2. Display
3. Search
4. Delete
5. Sort
6. Exit
Enter your choice: 2
Array: 54 12 1 7 20

1. Insert
2. Display
3. Search
4. Delete
5. Sort
6. Exit
Enter your choice: 5
Array sorted in ascending order.

1. Insert
2. Display
3. Search
4. Delete
5. Sort
6. Exit
Enter your choice: 2
Array: 1 7 12 20 54

1. Insert
2. Display
3. Search
4. Delete
5. Sort
6. Exit
Enter your choice: 3
Enter the value to search: 12
Element found at position 3.

1. Insert
2. Display
3. Search
4. Delete
5. Sort
6. Exit
Enter your choice: 4
Enter the value to delete: 1
First occurrence deleted.

1. Insert
2. Display
3. Search
4. Delete
5. Sort
6. Exit
```

LEARN • SERVE • EXCEL
RAJAGIRI

```
Enter your choice: 2
Array: 7 12 20 54

1. Insert
2. Display
3. Search
4. Delete
5. Sort
6. Exit
Enter your choice: 6
Program ended.
```

Program No: 3	**Date:** 20-07-2026

Program Title: Use a menu-driven program to insert, search, delete and sort elements in an array using functions (use only local variables)

Description:

Implements a menu-driven program to perform insertion, searching, deletion, and sorting on an array using local variables passed to functions.

Author: Abhishek Vincent | **Roll No:** 03 | **Date:** 20-07-2026

Source Code:

```
#include <stdio.h>

#define MAX 100

void display(int array[], int size)
{
   int i;

   if (size == 0)
   {
      printf("Array is empty.\n");
      return;
   }

   printf("Array: ");
   for (i = 0; i < size; i++)
   {
      printf("%d ", array[i]);
   }
   printf("\n");
}

int insert_element(int array[], int size)
{
   int value;
   int position;
   int i;

   if (size == MAX)
   {
      printf("Array is full.\n");
      return size;
   }

   printf("Enter the value to insert: ");
```

```
    scanf("%d", &value);
    printf("Enter the position (1 to %d): ", size + 1);
    scanf("%d", &position);

    if (position < 1 || position > size + 1)
    {
        printf("Invalid position.\n");
        return size;
    }

    for (i = size; i >= position; i--)
    {
        array[i] = array[i - 1];
    }

    array[position - 1] = value;
    printf("Element inserted.\n");
    return size + 1;
}

void search_element(int array[], int size)
{
    int value;
    int i;

    printf("Enter the value to search: ");
    scanf("%d", &value);

    for (i = 0; i < size; i++)
    {
        if (array[i] == value)
        {
            printf("Element found at position %d.\n", i + 1);
            return;
        }
    }

    printf("Element not found.\n");
}

int delete_element(int array[], int size)
{
    int value;
    int position = -1;
    int i;
```

LEARN • SERVE • EXCEL RAJAGIRI

```
    if (size == 0)
    {
        printf("Array is empty.\n");
        return size;
    }

    printf("Enter the value to delete: ");
    scanf("%d", &value);

    for (i = 0; i < size; i++)
    {
        if (array[i] == value)
        {
            position = i;
            break;
        }
    }

    if (position == -1)
    {
        printf("Element not found.\n");
        return size;
    }

    for (i = position; i < size - 1; i++)
    {
        array[i] = array[i + 1];
    }

    printf("First occurrence deleted.\n");
    return size - 1;
}

void sort_elements(int array[], int size)
{
    int i;
    int j;
    int temporary;

    for (i = 0; i < size - 1; i++)
    {
        for (j = 0; j < size - i - 1; j++)
        {
            if (array[j] > array[j + 1])
            {
                temporary = array[j];
```

```
            array[j] = array[j + 1];
            array[j + 1] = temporary;
        }
      }
    }

    printf("Array sorted in ascending order.\n");
}

int main(void)
{
    int array[MAX];
    int size = 0;
    int choice;

    do
    {
      printf("\n1. Insert\n");
      printf("2. Display\n");
      printf("3. Search\n");
      printf("4. Delete\n");
      printf("5. Sort\n");
      printf("6. Exit\n");
      printf("Enter your choice: ");
      scanf("%d", &choice);

      switch (choice)
      {
        case 1:
          size = insert_element(array, size);
          break;
        case 2:
          display(array, size);
          break;
        case 3:
          search_element(array, size);
          break;
        case 4:
          size = delete_element(array, size);
          break;
        case 5:
          sort_elements(array, size);
          break;
        case 6:
          printf("Program ended.\n");
          break;
```

```
        default:
            printf("Invalid choice.\n");
      }
   } while (choice != 6);

   return 0;
}
```

Output Text:

```
1. Insert
2. Display
3. Search
4. Delete
5. Sort
6. Exit
Enter your choice: 1
Enter the value to insert: 15
Enter the position (1 to 1): 1
Element inserted.

1. Insert
2. Display
3. Search
4. Delete
5. Sort
6. Exit
Enter your choice: 1
Enter the value to insert: 100
Enter the position (1 to 2): 2
Element inserted.

1. Insert
2. Display
3. Search
4. Delete
5. Sort
6. Exit
Enter your choice: 1
Enter the value to insert: 75
Enter the position (1 to 3): 3
Element inserted.

1. Insert
```

LEARN • SERVE • EXCEL RAJAGIRI

2. Display
3. Search
4. Delete
5. Sort
6. Exit
Enter your choice: 1
Enter the value to insert: 1
Enter the position (1 to 4): 4
Element inserted.

1. Insert
2. Display
3. Search
4. Delete
5. Sort
6. Exit
Enter your choice: 1
Enter the value to insert: 67
Enter the position (1 to 5): 5
Element inserted.

1. Insert
2. Display
3. Search
4. Delete
5. Sort
6. Exit
Enter your choice: 2
Array: 15 100 75 1 67

1. Insert
2. Display
3. Search
4. Delete
5. Sort
6. Exit
Enter your choice: 5
Array sorted in ascending order.

1. Insert
2. Display
3. Search
4. Delete
5. Sort
6. Exit
Enter your choice: 2

Array: 1 15 67 75 100

1. Insert
2. Display
3. Search
4. Delete
5. Sort
6. Exit
Enter your choice: 3
Enter the value to search: 67
Element found at position 3.

1. Insert
2. Display
3. Search
4. Delete
5. Sort
6. Exit
Enter your choice: 4
Enter the value to delete: 67
First occurrence deleted.

1. Insert
2. Display
3. Search
4. Delete
5. Sort
6. Exit
Enter your choice: 2
Array: 1 15 75 100

1. Insert
2. Display
3. Search
4. Delete
5. Sort
6. Exit
Enter your choice: 6
Program ended.

Program No: 4	**Date:** 22-07-2026

Program Title: Search for all the occurrences of an element in an integer array (positions)

Description:
Reads an array and searches for all positions where a given target element occurs.

Author: Abhishek Vincent | **Roll No:** 03 | **Date:** 22-07-2026

Source Code:

```
#include <stdio.h>

#define MAX 100

void read_array(int array[], int size)
{
   int i;

   for (i = 0; i < size; i++)
   {
      printf("Enter element %d: ", i + 1);
      scanf("%d", &array[i]);
   }
}

void display_occurrences(int array[], int size, int value)
{
   int i;
   int found = 0;

printf("Positions of %d: ", value);
   for (i = 0; i < size; i++)
   {
      if (array[i] == value)
      {
         printf("%d ", i + 1);
         found = 1;
      }
   }

   if (found == 0)
   {
      printf("Element not found");
   }
   printf("\n");
}
```

```
int main(void)
{
   int array[MAX];
   int size;
   int value;

   printf("Enter the size of the array (maximum %d): ", MAX);
   scanf("%d", &size);

   if (size < 1 || size > MAX)
   {
      printf("Invalid size.\n");
      return 0;
   }

   read_array(array, size);
   printf("Enter the element to search: ");
   scanf("%d", &value);
   display_occurrences(array, size, value);

   return 0;
}
```

Output Text:

```
Enter the size of the array (maximum 100): 5
Enter element 1: 32
Enter element 2: 1
Enter element 3: 41
Enter element 4: 23
Enter element 5: 1
Enter the element to search: 23
Positions of 23: 4
```

Program No: 5	**Date:** 22-07-2026

Program Title: Sort the array elements in ascending order (minimum three functions: read, disp and sort)

Description:

Sorts an array of integers in ascending order using functions for reading, displaying, and sorting elements.

Author: Abhishek Vincent | **Roll No:** 03 | **Date:** 22-07-2026

Source Code:

```
#include<stdio.h>

 int a[10],n=0;
 void read()
 {
        printf("\nEnter a elemnt:\t");
         scanf("%d",&a[n]);
         n++;
 }

 void disp()
 {
        int i;
        if(n==0)
        {
                printf("\n___NO ELEMENT TO DISPLAY___\n\n");
         }else
         {

        for(i=0;i<n;i++)
         {
                printf("%d\n",a[i]);
         }
 }
}
 void search()
 {
        int s,i,flag=0;
        printf("Enter element to search:\t");
        scanf("%d",&s);
        for(i=0;i<n;i++)
        {
                if(a[i]==s)
                {
                        printf("%d found at position %d\n\n",s,i+1);
                        flag=1;
```

```
                }

        }
        if(flag==0)
        {
                printf("%d Not found\n\n",s);

        }

}
void deletee()
{
        int i, j, del, flag=0;
        printf("Enter the element to delete:\t");
        scanf("%d",&del);
        for(i=0;i<n;i++)
        {
                if(a[i]==del)
                {
                        for(j=i;j<n-1;j++)
                        {
                                a[j]=a[j+1];
                        }
                        n--;
                        flag=1;
                        printf("%d Deleted\n\n",del);
                        break;
                }
        }
        if(flag==0)
{
        printf("%d Not found\n\n",del);
}
}
void sort()
{
        int i,j,temp;
 for(i=0;i<n-1;i++)
 {
        for(j=i+1;j<n;j++)
        {
                if(a[i]>a[j])
                {
                        temp=a[i];
                        a[i]=a[j];
                        a[j]=temp;
```

```
        }
      }
 }
 for(i=0;i<n;i++)
 {
        printf("%d\n",a[i]);
 }
}
 int main()
 {
                int ch;
        do{
        printf(" 1.Insert \n 2.Display \n 3.Search \n 4.Delete \n 5.Sort \n 6.Exit \n\n\nEnter a
Option:\t");
        scanf("%d",&ch);
         switch(ch)
         {
                case 1: read();
                break;
                case 2: disp();
                break;
                case 3: search();
                break;
    case 4: deletee();
        break;
    case 5: sort();
    break;
        case 6: printf("Exiting.....");
                break;
                default: printf("_____Invalid Option_____");
         }
         } while(ch != 6);
         return 0;
 }
```

Output Text:

```
1.Insert
 2.Display
 3.Search
 4.Delete
 5.Sort
 6.Exit
Enter a Option: 1
```

```
Enter a elemnt:43
 1.Insert
 2.Display
 3.Search
 4.Delete
 5.Sort
 6.Exit
Enter a Option: 1

Enter a elemnt:6
 1.Insert
 2.Display
 3.Search
 4.Delete
 5.Sort
 6.Exit
Enter a Option: 1

Enter a elemnt:23
 1.Insert
 2.Display
 3.Search
 4.Delete
 5.Sort
 6.Exit
Enter a Option: 1

Enter a elemnt:78
 1.Insert
 2.Display
 3.Search
 4.Delete
 5.Sort
 6.Exit
Enter a Option: 2
43      6       23      78
1.Insert
 2.Display
 3.Search
 4.Delete
 5.Sort
 6.Exit
Enter a Option: 5
6       23      43      78
1.Insert
 2.Display
```

```
 3.Search
 4.Delete
 5.Sort
 6.Exit
Enter a Option:3
Enter element to search: 23
23 found at position 2
 1.Insert
 2.Display
 3.Search
 4.Delete
 5.Sort
 6.Exit
Enter a Option:4
Enter the element to delete:     6
6 Deleted

 1.Insert
 2.Display
 3.Search
 4.Delete
 5.Sort
 6.Exit
Enter a Option:2
23       43       78
1.Insert
 2.Display
 3.Search
 4.Delete
 5.Sort
 6.Exit
Enter a Option:6
Exiting.....⏎
```

Program No: 6	**Date:** 02-08-2026

Program Title: Display the array elements in the same order using a recursive function

Description:
Prints the elements of an integer array in their original order using recursion.
Author: Abhishek Vincent | **Roll No:** 03 | **Date:** 02-08-2026

Source Code:

```
#include <stdio.h>

#define MAX 100

void read_array(int array[], int size)
{
   int i;

   for (i = 0; i < size; i++)
   {
      printf("Enter element %d: ", i + 1);
      scanf("%d", &array[i]);
   }
}

void display_in_same_order(int array[], int size, int position)
{
   if (position == size)
   {
      return;
   }

   printf("%d ", array[position]);
   display_in_same_order(array, size, position + 1);
}

int main()
{
   int array[MAX];
   int size;

   printf("Enter the size of the array (maximum %d): ", MAX);
   scanf("%d", &size);

   if (size < 1 || size > MAX)
   {
      printf("Invalid size.\n");
```

```
    return 0;
  }

  read_array(array, size);

  printf("Array in the same order: ");
  display_in_same_order(array, size, 0);
  printf("\n");

  return 0;
}
```

Output Text:

```
Enter the size of the array (maximum 100): 5
Enter element 1: 23
Enter element 2: 51
Enter element 3: 2
Enter element 4: 67
Enter element 5: 45
Array in the same order: 23 51 2 67 45
```

LEARN • SERVE • EXCEL
RAJAGIRI

Program No: 7	**Date:** 02-08-2026

Program Title: Display array elements in reverse order using a recursive function .

Description:
Prints the elements of an integer array in reverse order using recursion.
Author: Abhishek Vincent | **Roll No:** 03 | **Date:** 02-08-2026

Source Code:

```
#include <stdio.h>

#define MAX 100

void read_array(int array[], int size)
{
   int i;

   for (i = 0; i < size; i++)
   {
      printf("Enter element %d: ", i + 1);
      scanf("%d", &array[i]);
   }
}

void display_in_reverse(int array[], int position)
{
   if (position < 0)
   {
      return;
   }

   printf("%d ", array[position]);
   display_in_reverse(array, position - 1);
}

int main()
{
   int array[MAX];
   int size;

   printf("Enter the size of the array (maximum %d): ", MAX);
   scanf("%d", &size);

   if (size < 1 || size > MAX)
```

```
    {
        printf("Invalid size.\n");
        return 0;
    }

    read_array(array, size);

    printf("Array in reverse order: ");
    display_in_reverse(array, size - 1);
    printf("\n");

    return 0;
}
```

Output Text:

```
Enter the size of the array (maximum 100): 5
Enter element 1: 32
Enter element 2: 13
Enter element 3: 52
Enter element 4: 11
Enter element 5: 34
Array in reverse order: 34 11 52 13 32
```

LEARN • SERVE • EXCEL RAJAGIRI

Program No: 8	**Date:** 28-07-2026

Program Title: Perform the following matrix operations

a. Adding two matrices
b. Subtraction of one matrix from another.
c. Multiplication of two matrices
d. Transpose of a matrix.
e. Determinant of a matrix(2x2 and 3x3)

Description:

Performs various matrix operations including matrix addition, subtraction, multiplication, transpose, and determinant calculation for 2x2 and 3x3 matrices.

Author: Abhishek Vincent | **Roll No:** 03 | **Date:** 28-07-2026

Source Code:

```
#include <stdio.h>

#define MAX 3

void read_matrix(int matrix[][MAX], int rows, int columns)
{
  int i;
  int j;

  for (i = 0; i < rows; i++)
  {
    for (j = 0; j < columns; j++)
    {
      printf("Enter element at row %d, column %d: ", i + 1, j + 1);
      scanf("%d", &matrix[i][j]);
    }
  }
}

void display_matrix(int matrix[][MAX], int rows, int columns)
{
  int i;
  int j;

  for (i = 0; i < rows; i++)
  {
```

```
      for (j = 0; j < columns; j++)
      {
         printf("%d ", matrix[i][j]);
      }
      printf("\n");
   }
}

void add_matrices()
{
   int first[MAX][MAX];
   int second[MAX][MAX];
   int result[MAX][MAX];
   int rows;
   int columns;
   int i;
   int j;

   printf("Enter rows and columns (maximum %d): ", MAX);
   scanf("%d %d", &rows, &columns);
   if (rows < 1 || rows > MAX || columns < 1 || columns > MAX)
   {
      printf("Invalid matrix size.\n");
      return;
   }

   printf("Enter the first matrix:\n");
   read_matrix(first, rows, columns);
   printf("Enter the second matrix:\n");
   read_matrix(second, rows, columns);

   for (i = 0; i < rows; i++)
   {
      for (j = 0; j < columns; j++)
      {
         result[i][j] = first[i][j] + second[i][j];
      }
   }

   printf("Sum of the matrices:\n");
   display_matrix(result, rows, columns);
}

void subtract_matrices(void)
{
   int first[MAX][MAX];
```

```
    int second[MAX][MAX];
    int result[MAX][MAX];
    int rows;
    int columns;
    int i;
    int j;

    printf("Enter rows and columns (maximum %d): ", MAX);
    scanf("%d %d", &rows, &columns);
    if (rows < 1 || rows > MAX || columns < 1 || columns > MAX)
    {
        printf("Invalid matrix size.\n");
        return;
    }

    printf("Enter the first matrix:\n");
    read_matrix(first, rows, columns);
    printf("Enter the second matrix:\n");
    read_matrix(second, rows, columns);

    for (i = 0; i < rows; i++)
    {
        for (j = 0; j < columns; j++)
        {
            result[i][j] = first[i][j] - second[i][j];
        }
    }

    printf("First matrix minus second matrix:\n");
    display_matrix(result, rows, columns);
}

void multiply_matrices(void)
{
    int first[MAX][MAX];
    int second[MAX][MAX];
    int result[MAX][MAX];
    int first_rows;
    int first_columns;
    int second_rows;
    int second_columns;
    int i;
    int j;
    int k;

    printf("Enter rows and columns of the first matrix: ");
```

```
    scanf("%d %d", &first_rows, &first_columns);
    printf("Enter rows and columns of the second matrix: ");
    scanf("%d %d", &second_rows, &second_columns);

    if (first_rows < 1 || first_rows > MAX || first_columns < 1 ||
        first_columns > MAX || second_rows < 1 || second_rows > MAX ||
        second_columns < 1 || second_columns > MAX)
    {
        printf("Invalid matrix size.\n");
        return;
    }

    if (first_columns != second_rows)
    {
        printf("Matrix multiplication is not possible.\n");
        return;
    }

    printf("Enter the first matrix:\n");
    read_matrix(first, first_rows, first_columns);
    printf("Enter the second matrix:\n");
    read_matrix(second, second_rows, second_columns);

    for (i = 0; i < first_rows; i++)
    {
        for (j = 0; j < second_columns; j++)
        {
            result[i][j] = 0;
            for (k = 0; k < first_columns; k++)
            {
                result[i][j] = result[i][j] +
                               first[i][k] * second[k][j];
            }
        }
    }

    printf("Product of the matrices:\n");
    display_matrix(result, first_rows, second_columns);
}

void transpose_matrix(void)
{
    int matrix[MAX][MAX];
    int rows;
    int columns;
    int i;
```

```
    int j;

    printf("Enter rows and columns (maximum %d): ", MAX);
    scanf("%d %d", &rows, &columns);
    if (rows < 1 || rows > MAX || columns < 1 || columns > MAX)
    {
        printf("Invalid matrix size.\n");
        return;
    }

    read_matrix(matrix, rows, columns);

    printf("Transpose of the matrix:\n");
    for (i = 0; i < columns; i++)
    {
        for (j = 0; j < rows; j++)
        {
            printf("%d ", matrix[j][i]);
        }
        printf("\n");
    }
}

void find_determinant(void)
{
    int matrix[MAX][MAX];
    int order;
    int determinant;

    printf("Enter the order of the matrix (2 or 3): ");
    scanf("%d", &order);
    if (order != 2 && order != 3)
    {
        printf("Only 2x2 and 3x3 matrices are supported.\n");
        return;
    }

    read_matrix(matrix, order, order);

    if (order == 2)
    {
        determinant = matrix[0][0] * matrix[1][1] -
                      matrix[0][1] * matrix[1][0];
    }
    else
    {
```

```
    determinant = matrix[0][0] *
                  (matrix[1][1] * matrix[2][2] -
                   matrix[1][2] * matrix[2][1]) -
                 matrix[0][1] *
                   (matrix[1][0] * matrix[2][2] -
                    matrix[1][2] * matrix[2][0]) +
                 matrix[0][2] *
                   (matrix[1][0] * matrix[2][1] -
                    matrix[1][1] * matrix[2][0]);
  }

  printf("Determinant = %d\n", determinant);
}

int main(void)
{
  int choice;

  do
  {
    printf("\n1. Add two matrices\n");
    printf("2. Subtract two matrices\n");
    printf("3. Multiply two matrices\n");
    printf("4. Transpose a matrix\n");
    printf("5. Find determinant\n");
    printf("6. Exit\n");
    printf("Enter your choice: ");
    scanf("%d", &choice);

    switch (choice)
    {
      case 1:
        add_matrices();
        break;
      case 2:
        subtract_matrices();
        break;
      case 3:
        multiply_matrices();
        break;
      case 4:
        transpose_matrix();
        break;
      case 5:
        find_determinant();
        break;
```

```
        case 6:
            printf("Program ended.\n");
            break;
        default:
            printf("Invalid choice.\n");
    }
  } while (choice != 6);

  return 0;
}
```

Output Text:

```
1. Add two matrices
2. Subtract two matrices
3. Multiply two matrices
4. Transpose a matrix
5. Find determinant
6. Exit
Enter your choice: 1
Enter rows and columns (maximum 3): 2 3
Enter the first matrix:
Enter element at row 1, column 1: 12
Enter element at row 1, column 2: 32
Enter element at row 1, column 3: 14
Enter element at row 2, column 1: 51
Enter element at row 2, column 2: 23
Enter element at row 2, column 3: 13
Enter the second matrix:
Enter element at row 1, column 1: 31
Enter element at row 1, column 2: 123
Enter element at row 1, column 3: 12
Enter element at row 2, column 1: 4
Enter element at row 2, column 2: 52
Enter element at row 2, column 3: 34
Sum of the matrices:
43 155 26
55 75 47

1. Add two matrices
2. Subtract two matrices
3. Multiply two matrices
4. Transpose a matrix
5. Find determinant
```

```
6. Exit

Enter your choice: 2
Enter rows and columns (maximum 3): 2 2
Enter the first matrix:
Enter element at row 1, column 1: 12
Enter element at row 1, column 2: 32
Enter element at row 2, column 1: 31
Enter element at row 2, column 2: 23
Enter the second matrix:
Enter element at row 1, column 1: 12
Enter element at row 1, column 2: 23
Enter element at row 2, column 1: 42
Enter element at row 2, column 2: 12
First matrix minus second matrix:
0 9
-11 11

1. Add two matrices
2. Subtract two matrices
3. Multiply two matrices
4. Transpose a matrix
5. Find determinant
6. Exit
Enter your choice: 3
Enter rows and columns of the first matrix: 2 2
Enter rows and columns of the second matrix: 2 2
Enter the first matrix:
Enter element at row 1, column 1: 31
Enter element at row 1, column 2: 2
Enter element at row 2, column 1: 41
Enter element at row 2, column 2: 23
Enter the second matrix:
Enter element at row 1, column 1: 12
Enter element at row 1, column 2: 32
Enter element at row 2, column 1: 2
Enter element at row 2, column 2: 12
Product of the matrices:
376 1016
538 1588

1. Add two matrices
2. Subtract two matrices
3. Multiply two matrices
4. Transpose a matrix
5. Find determinant
```

```
6. Exit
Enter your choice: 4
Enter rows and columns (maximum 3): 2 2
Enter element at row 1, column 1: 12
Enter element at row 1, column 2: 31
Enter element at row 2, column 1: 31
Enter element at row 2, column 2: 23
Transpose of the matrix:
12 31
31 23

1. Add two matrices
2. Subtract two matrices
3. Multiply two matrices
4. Transpose a matrix
5. Find determinant
6. Exit
Enter your choice: 5
Enter the order of the matrix (2 or 3): 2
Enter element at row 1, column 1: 12
Enter element at row 1, column 2: 31
Enter element at row 2, column 1: 3
Enter element at row 2, column 2: 12
Determinant = 51

1. Add two matrices
2. Subtract two matrices
3. Multiply two matrices
4. Transpose a matrix
5. Find determinant
6. Exit
Enter your choice: 6
Program ended.
```

LEARN • SERVE • EXCEL
RAJAGIRI

Program No: 9	**Date:** 24-07-2026

Program Title: Read a String and display it in the reverse order using the same array itself

Description:

Reads a string from the user and reverses it in-place using the same character array.

Author: Abhishek Vincent | **Roll No:** 03 | **Date:** 24-07-2026

Source Code:

```
#include <stdio.h>

#define MAX_LENGTH 100

void read_string(char text[])
{
   printf("Enter a string: ");
   scanf(" %99[^\n]", text);
}

void reverse_string(char text[])
{
   int length = 0;
   int left;
   int right;
   char temporary;

   while (text[length] != '\0')
   {
      length++;
   }

   left = 0;
   right = length - 1;
   while (left < right)
   {
      temporary = text[left];
      text[left] = text[right];
      text[right] = temporary;
      left++;
      right--;
   }
}
```

```
int main()
{
   char text[MAX_LENGTH];

   read_string(text);
   reverse_string(text);
   printf("String in reverse order: %s\n", text);

   return 0;
}
```

Output Text:

```
Enter a string: abhishek
String in reverse order: kehsihba
```

LEARN • SERVE • EXCEL
RAJAGIRI

Program No: 10	**Date:** 24-07-2026

Program Title: Read n Strings and display them in the ascending order.

Description:
Reads a string and displays them in the ascending order
Author: Abhishek Vincent | **Roll No:** 03 | **Date:** 24-07-2026

Source Code:

```
#include <stdio.h>

#define MAX_STRINGS 20
#define MAX_LENGTH 50

int read_strings(char strings[][MAX_LENGTH])
{
  int count;
  int i;

  printf("Enter the number of strings (maximum %d): ", MAX_STRINGS);
  scanf("%d", &count);

  if (count < 1 || count > MAX_STRINGS)
  {
    return 0;
  }

  for (i = 0; i < count; i++)
  {
    printf("Enter string %d: ", i + 1);
    scanf(" %49[^\n]", strings[i]);
  }

  return count;
}

int compare_strings(char first[], char second[])
{
  int i = 0;

  while (first[i] != '\0' && second[i] != '\0')
  {
```

```
      if (first[i] < second[i])
      {
         return -1;
      }
      if (first[i] > second[i])
      {
         return 1;
      }
      i++;
   }

   if (first[i] == '\0' && second[i] == '\0')
   {
      return 0;
   }
   if (first[i] == '\0')
      return -1;
   }
   return 1;
}

void sort_strings(char strings[][MAX_LENGTH], int count)
{
   int i;
   int j;
   int k;
   char temporary;

   for (i = 0; i < count - 1; i++)
   {
      for (j = 0; j < count - i - 1; j++)
      {
         if (compare_strings(strings[j], strings[j + 1]) > 0)
         {
            for (k = 0; k < MAX_LENGTH; k++)
            {
               temporary = strings[j][k];
               strings[j][k] = strings[j + 1][k];
               strings[j + 1][k] = temporary;
            }
         }
      }
   }
}

void display_strings(char strings[][MAX_LENGTH], int count)
```

```
{
   int i;

   for (i = 0; i < count; i++)
   {
      printf("%s\n", strings[i]);
   }
}

int main()
{
   char strings[MAX_STRINGS][MAX_LENGTH];
   int count;

   count = read_strings(strings);
   if (count == 0)
   {
      printf("Invalid number of strings.\n");
      return 0;
   }

   sort_strings(strings, count);

   printf("Strings in ascending order:\n");
   display_strings(strings, count);

   return 0;
}
```

Output Text:

```
Enter the number of strings (maximum 20): 3
Enter string 1: sterling
Enter string 2: lachmiAmmal
Enter string 3: Abhizeak
Strings in ascending order:
Abhizeak
lachmiAmmal
sterling
```